

Weather Service – Semantic Weather Search and RAG Assistant

This project extends the Databricks App sample into a semantic weather intelligence application using **Lakebase**, **pgvector**, **Sentence Transformers**, and optional **Retrieval-Augmented Generation (RAG)** capabilities.

The application ingests weather information from the **National Weather Service (NWS)** API, normalizes the data into searchable documents, generates vector embeddings, and enables natural-language semantic search over weather information.

The application supports:

- Weather document ingestion
- Vector-based semantic search
- Source filtering
- Scheduled synchronization
- pgvector similarity search

Data Source

This project uses the **U.S. National Weather Service (NWS) API**:

```
https://api.weather.gov
```

Why the National Weather Service?

The NWS API was selected because it:

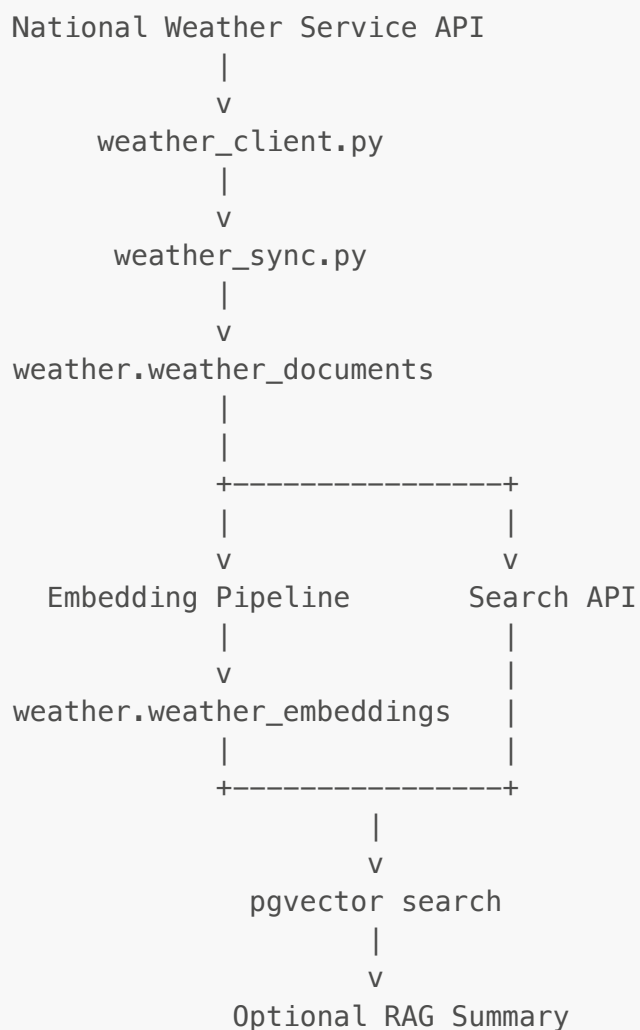
- Is the official weather API provided by the U.S. government.
- Does not require API keys or authentication.
- Provides authoritative weather alerts and forecast information.
- Returns structured JSON suitable for ingestion pipelines.
- Is free to use for educational and demonstration purposes.

The application currently supports:

- Weather alerts
- Forecast discussions
- Forecast information

Each response is normalized into a common document format before being stored in Lakebase.

Architecture Overview



Data Model

The application uses three primary tables.

weather.location_cache

Caches geolocation and NWS grid information.

Purpose:

- Avoid repeated location lookups.
- Reduce unnecessary API requests.

Example columns:

- location_name

- latitude
- longitude
- grid_x
- grid_y
- office
- updated_at

weather.weather_documents

Stores normalized weather documents.

Each row represents a searchable weather document.

Important columns:

Column	Description
id	Deterministic document identifier
location	Human-readable location
source_type	Weather document category
headline	Weather headline
narrative_text	Searchable weather content
issued_at	Document issue timestamp
effective_at	Effective start time
payload	Original API JSON
content_hash	Used for change detection
synced_at	Last synchronization timestamp

Supported **source_type** values:

```
alert
forecast
observation
```

Idempotent ingestion

Weather synchronization uses deterministic document IDs and PostgreSQL upserts.

Example behavior:

Running:

```
POST /weather/sync
```

multiple times will:

- Update changed documents.
- Avoid duplicate rows.
- Preserve document identity.

weather.weather_embeddings

Stores vector embeddings generated from weather document chunks.

Columns:

Column	Description
id	Unique embedding identifier
document_id	Parent weather document
chunk_index	Chunk sequence number
chunk_text	Embedded text
embedding	pgvector VECTOR(384)
model_name	Embedding model
created_at	Creation timestamp

The application uses:

```
sentence-transformers/all-MiniLM-L6-v2
```

with:

```
384 dimensions
```

The embedding table uses pgvector for similarity search.

Embedding Strategy

The same embedding model used by the Day-2 news semantic search pipeline is used here.

Model:

```
sentence-transformers/all-MiniLM-L6-v2
```

Dimension:

```
384
```

Reasons:

- Good semantic performance for short documents.
- Fast inference.
- Compatible with existing vector search implementations.
- Small enough for interactive applications.

Chunking Strategy

Weather documents are chunked before embedding.

Configuration:

```
CHUNK_SIZE = 800  
CHUNK_OVERLAP = 100
```

The application uses a sliding-window approach.

Most weather alerts and forecasts are short enough to fit in one chunk.

Chunking primarily helps with:

- Long flood alerts.
- Severe weather instructions.
- Forecast discussions.
- Combined weather narratives.

End-to-End Pipeline

The application consists of four stages.

1. Synchronize Weather Data

The Databricks App retrieves weather information from NWS.

Endpoint:

```
POST /weather/sync
```

Example:

```
{
  "locations": [
    "Chicago, IL",
    "Austin, TX"
  ]
}
```

The data is stored in:

```
weather.weather_documents
```

Supported sources:

```
alert
forecast
observation
```

2. Generate Embeddings

Run:

```
notebooks/ingest_weather_embeddings.py
```

The embedding pipeline:

1. Reads weather documents.
2. Detects previously embedded documents.
3. Splits large documents into chunks.
4. Generates Sentence Transformer embeddings.
5. Writes vectors using psycopg and pgvector.

Output:

```
weather.weather_embeddings
```

3. Semantic Search

The application supports both POST and GET search.

POST

Endpoint:

```
POST /weather/search
```

Request:

```
{
  "query": "risk of flooding near rivers",
  "top_k": 5
}
```

GET

Example:

```
GET /weather/search?query=flood risk
```

Optional filters:

```
source_type=alert
```

Example:

```
GET /weather/search?
query=flood risk
&source_type=alert
```

Search process:

1. Embed the user query.
2. Execute pgvector cosine similarity search.
3. Return the most relevant weather chunks.

Example:

```
{
  "query": "flood risk near rivers",
  "results": [
    {
      "location": "Chicago, IL",
      "headline": "Flood Warning",
      "source_type": "alert",
      "chunk_text": "...",
      "similarity": 0.92
    }
  ]
}
```

4. Optional RAG Weather Summary

The application (in future) can optionally generate a natural-language summary from retrieved results.

Flow:

```
User Query
  |
  v
Embedding Search
  |
  v
Top Weather Documents
  |
  v
LLM Summary
```

Example:

```
Several flood warnings are active near Chicago.
Residents near rivers should monitor local alerts
and avoid flooded roadways.
```

The LLM layer is designed to support:

- Databricks Foundation Model APIs

- OpenAI-compatible endpoints
- Other enterprise LLM services

Vector Search Optimization

The embedding table supports pgvector HNSW indexing.

Index:

```
CREATE INDEX weather_embeddings_hnsw_idx

ON weather.weather_embeddings

USING hnsw

(
    embedding vector_cosine_ops
);
```

Benefits:

- Faster nearest-neighbor searches.
- Better scalability as embeddings grow.
- Reduced query latency compared with sequential scans.

Benchmark queries are provided under:

```
sql/05_vector_benchmark.sql
```

Scheduled Synchronization

The application supports scheduled refresh using Databricks Jobs.

Example:

```
Every 15 minutes
```

Typical production flow:

```
Databricks Job
|
```

```
      v
sync_weather_documents.py
      |
      v
weather.weather_documents
      |
      v
ingest_weather_embeddings.py
```

Technologies Used

- Databricks Apps
- Databricks Asset Bundles
- Lakebase PostgreSQL
- pgvector
- psycopg
- Flask
- Sentence Transformers
- PyTorch
- National Weather Service API

Known Limitations

Current limitations:

- The application currently supports U.S. locations only because it relies on NWS.
- Weather information becomes stale without scheduled synchronization.
- Semantic search currently focuses on vector similarity.
- Hybrid keyword + vector search is not implemented.
- LLM summaries depend on an external model endpoint.
- Weather prediction is not performed; the system only retrieves and summarizes available information.

Future Improvements

Potential enhancements:

- Add hybrid search combining:
 - pgvector similarity
 - PostgreSQL full-text search
 - Optional LLM-generated weather summaries

- Add weather severity filtering:
 - tornado
 - flood
 - hurricane
 - winter storm
 - Add geographic filtering:
 - state
 - county
 - radius search
 - Add additional providers:
 - NOAA datasets
 - commercial weather APIs
 - radar feeds
 - Add automated embedding refresh only for changed documents.
 - Add cross-encoder reranking for improved retrieval accuracy.
 - Add conversational weather assistant capabilities.
-

Repository Components

```
./
├── app.py
├── weather_client.py
├── weather_sync.py
├── weather_search.py
├── weather_schema.py
├── lakebase.py
├── notebooks/
│   ├── sync_weather_documents.py
│   └── ingest_weather_embeddings.py
├── resources/
│   ├── sync_weather_documents_job.yml
│   └── ingest_weather_embeddings_job.yml
├── sql/
│   ├── 00_create_schema.sql
│   ├── 01_create_location_cache.sql
│   ├── 02_create_weather_documents.sql
│   ├── 03_create_embedding_tables.sql
│   ├── 04_create_indexes.sql
│   └── 05_seed_default_locations.sql
```

- |—— 06_vector_benchmark_before_hnsw_idx.sql
- |—— 07_create_hnsw_index.sql
- |—— 08_vector_benchmark_after_hnsw_idx.sql
- |—— 99_cleanup.sql
- |—— README.md

- |—— README.md
- |—— README_WEATHER.md